

Chapter 6: Concurrency Mastery: Asyncio, Threading & Multiprocessing

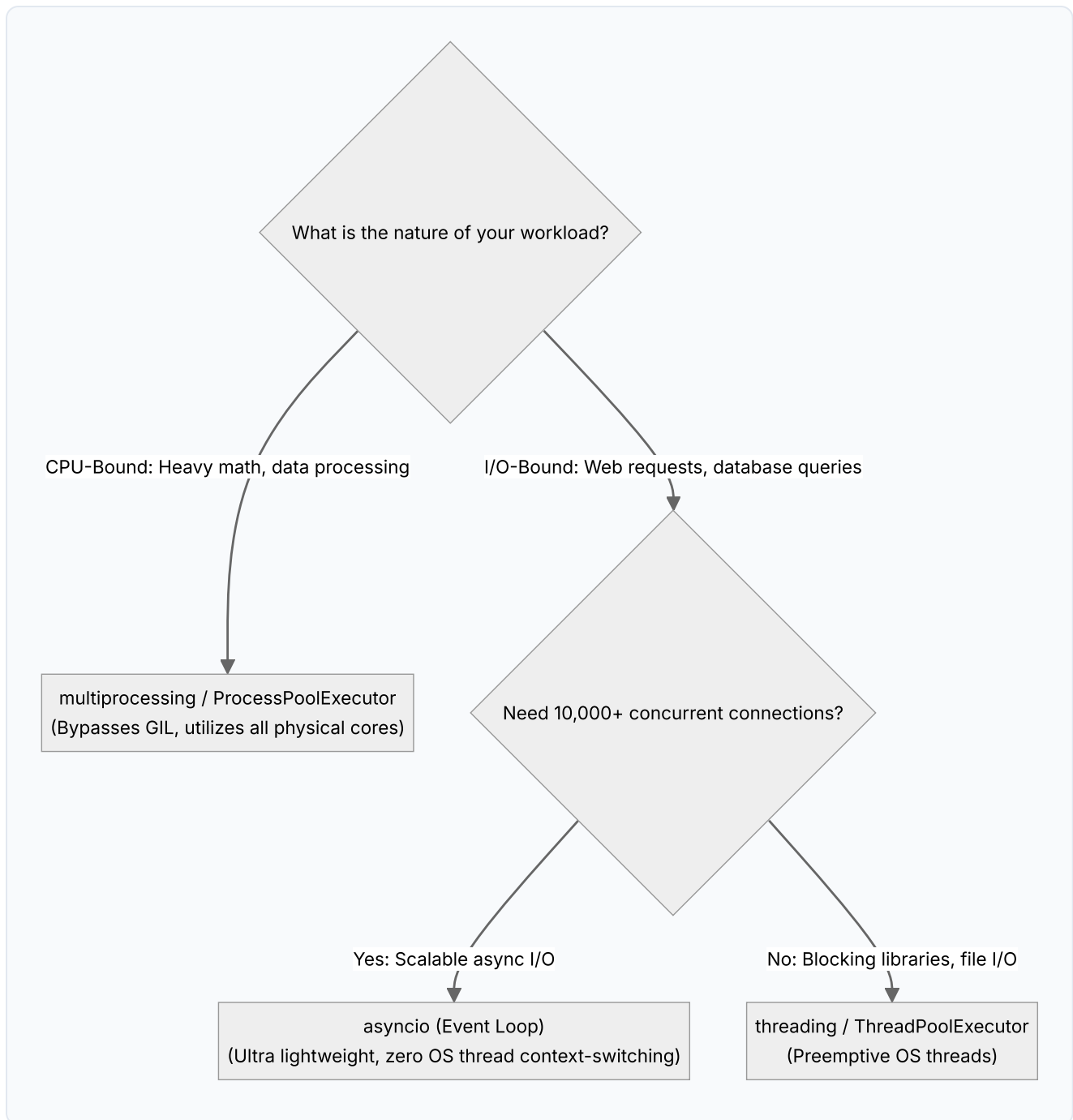
Core Learning Objective: Master the three fundamental paradigms of concurrency in Python. Understand OS threads and lock contention, cooperative single-threaded asynchronous I/O via the event loop, and true multi-core parallel processing with Inter-Process Communication (IPC).

1. The Concurrency Landscape: Choosing the Right Engine

Zero-Prerequisite Intuition: The "Single Master Chef vs. Multiple Kitchens" Metaphor What is the intuitive difference between `asyncio`, `threading`, and `multiprocessing`?

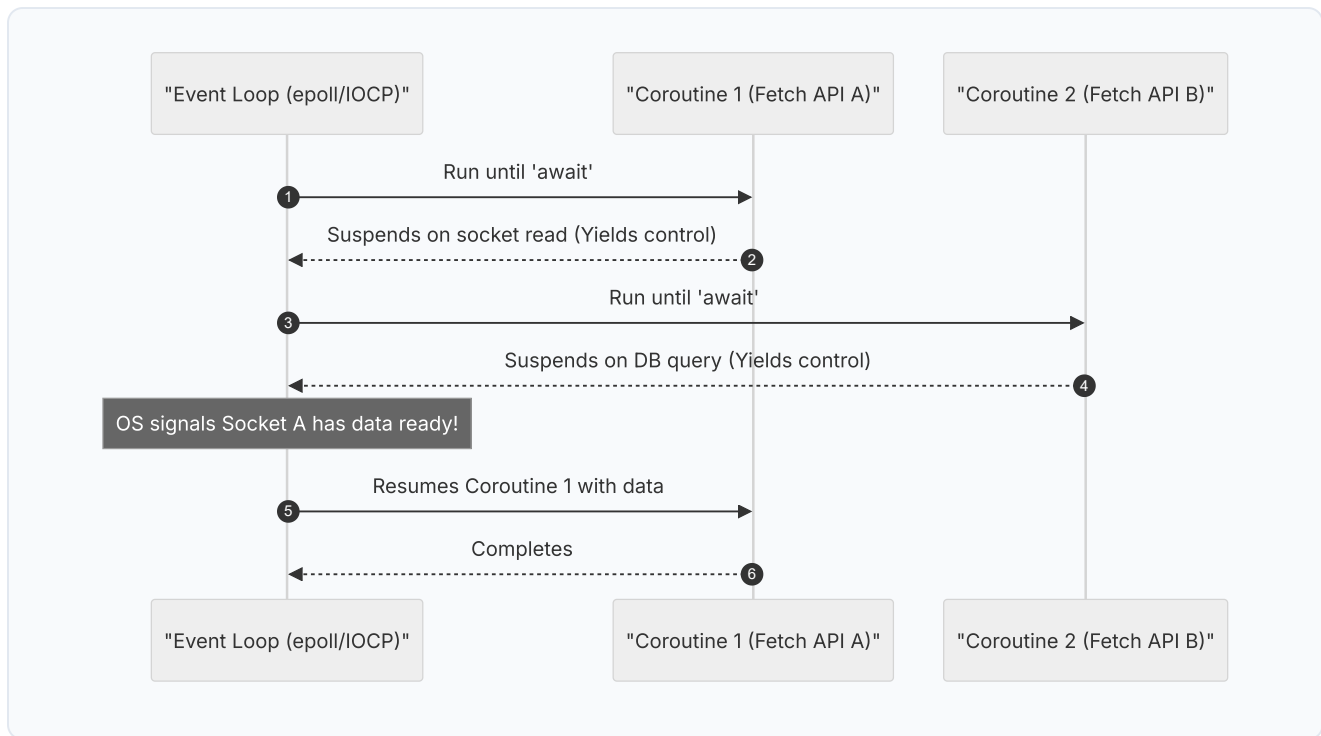
- **`asyncio` is One Master Chef in One Kitchen:** The chef puts pasta in a pot to boil. Instead of standing there staring at the boiling water for 10 minutes (blocking), the chef immediately turns around to dice carrots and chop onions. When the kitchen timer rings (`await`), the chef pivots back to drain the pasta. One person handles enormous volumes of orders because they never waste time standing idle! *(Best for 10,000+ network requests and database queries).*
- **`threading` is Three Cooks in One Kitchen with ONE Spoon (The GIL):** You hire three cooks in a single kitchen, but company policy says only ONE cook may hold the wooden spoon at any instant. Cook 1 stirs for a second, the manager shouts "Time's up!", and Cook 2 grabs the spoon. They waste substantial energy bumping into each other and negotiating who gets the spoon next. *(Best for blocking legacy I/O libraries).*
- **`multiprocessing` is Building Three Completely Separate Kitchens:** Each kitchen has its own chef, its own pantry, its own stove, and its own spoons. They can roast three whole turkeys completely in parallel at maximum physical speed without ever blocking each other. *(Best for heavy math, image crunching, and ML workloads).*

Concurrency Model	Module	Best Suited For	Concurrency Mechanism	CPU Cores Utilized	Communication Cost
Cooperative Coroutines	asyncio	High-volume network I/O (10,000+ concurrent sockets, WebSockets, microservices)	Single-threaded event loop multiplexing non-blocking I/O (<code>epoll</code> / <code>IOCP</code>)	1 Core	Near zero (Shared heap)
Preemptive Threads	threading	I/O-bound tasks with blocking legacy libraries, file operations, small thread counts	OS-level threads scheduled by OS kernel (GIL held during execution)	1 Core (at a time)	Low (Shared heap with lock synchronization)
Parallel Processes	multiprocessing	CPU-intensive computing (machine learning, data processing, crypto hashing)	Separate OS processes, each with its own independent Python interpreter and GIL	All Available Cores	High (Requires IPC / pickling serialization)



2. Asynchronous I/O with `asyncio` Under the Hood

`asyncio` does not create operating system threads. It executes on a **single OS thread** powered by an **Event Loop**.



The Three Pillars of `asyncio` :

1. **Coroutine Function (`async def`)**: Calling it does *not* execute code immediately; it instantiates a coroutine object.
2. **`await` keyword**: Yields execution back to the event loop until the awaited `Future` resolves.
3. **`asyncio.Task`** : Wraps a coroutine into a background schedule on the event loop via `asyncio.create_task()`.

Modern Structured Concurrency with `asyncio.TaskGroup` (Python 3.11+)

Prior to Python 3.11, developers used `asyncio.gather()`, which lacked robust error isolation. If one task crashed, sibling tasks were orphaned. Modern Python provides `TaskGroup` :

Controlling Rate & Concurrency: `asyncio.Semaphore`

When querying external APIs, unbounded concurrency will trigger rate limits or exhaust socket file descriptors:

3. Preemptive Threading: Synchronization & Race Conditions

When multiple OS threads access shared mutable objects, non-atomic operations lead to catastrophic **Race Conditions**.

Essential Thread Synchronization Primitives

1. Mutual Exclusion: `threading.Lock`

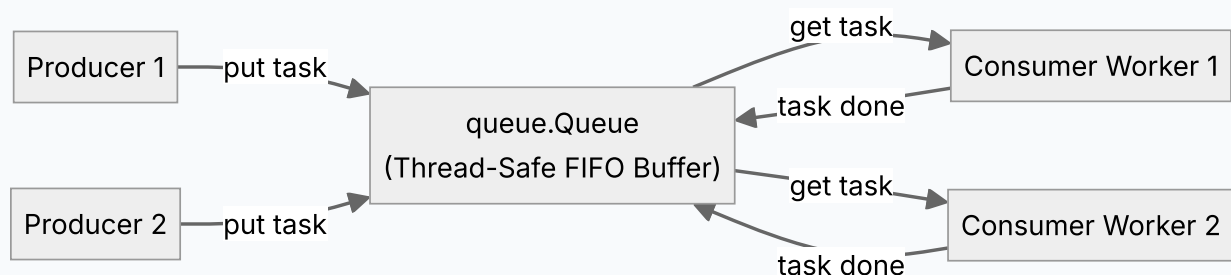
2. Reentrant Lock: `threading.RLock`

A standard `Lock` cannot be acquired twice by the same thread—attempting to do so causes an immediate **deadlock**. An `RLock` tracks ownership and allows the owning thread to acquire it multiple times recursively:

4. Multi-Process Parallelism: Bypassing the GIL

To execute compute-intensive code across physical CPU cores, instantiate separate processes using `concurrent.futures.ProcessPoolExecutor`:

5. Enterprise Pattern: Thread-Safe Producer-Consumer Pipeline



Version Notes

- `asyncio.TaskGroup`, `asyncio.timeout()` and `except*` exception groups are 3.11+. Before 3.11 use `gather` and `wait_for`.
- Free-threaded CPython (no GIL) is an optional build: experimental in 3.13 (PEP 703), officially supported but not the default in 3.14 (PEP 779). Check `sys._is_gil_enabled()` before assuming either.
- The default `multiprocessing` start method on POSIX changes from `fork` to `forkserver` in 3.14; code that relied on inheriting globals through `fork` must pass state explicitly.

Exercises

Three graded exercises for this chapter (two coding, one debugging) with hidden tests:



Attempt first; the reference solutions are in `exercises/_answers/ch06.py`.

Further Reading

- [asyncio documentation](#)
- [threading module](#)
- [concurrent.futures](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▼ 1. For a CPU-bound task on a standard CPython build, threads or processes?

Processes (`multiprocessing` / `ProcessPoolExecutor`), because the GIL lets only one thread run Python bytecode at a time. Threads suit I/O-bound work.

- ▼ 2. What does `await` do that a blocking call does not?

It suspends the current coroutine and returns control to the event loop so other tasks run, then resumes when the awaited operation completes.

- ▼ 3. Why does a `time.sleep(5)` inside `async def` freeze every other task?

It blocks the single event-loop thread. Use `await asyncio.sleep(5)` or move blocking work to an executor (`asyncio.to_thread`).

- ▼ 4. What does `asyncio.TaskGroup` give you over `gather`?

Structured concurrency: if one task fails the others are cancelled and the errors are raised together as an `ExceptionGroup`, so no task is left running unobserved.